Project Coding – Backend source (Go)

Seminar Hall Booking System – key Go segments (auth, requests, approval; full repo on GitHub)

Full source code – Included the most important code segments only (core schema, routing, JWT auth, request creation, department approval, overlap checks, and key UI logic). Omitted lines are marked with `// ...` or `-- ....` Full sources are on GitHub. Repository: https://github.com/sreejithr247/seminar-hall-booking-system.

backend/main.go

```
package main

import (
	"log"
	"os"

	"seminar-hall-booking-system/internal/config"
	"seminar-hall-booking-system/internal/database"
	"seminar-hall-booking-system/internal/router"

	"github.com/gin-gonic/gin"
	"github.com/joho/godotenv"
)

func main() {
	// Load environment variables (.env only — not .env.production; that file is for deploy docs / hosts)
	_ = godotenv.Load(".env")
	_ = godotenv.Overload(".env.local")

	// Load configuration
	cfg := config.Load()

	// Initialize database connection
	db, err := database.NewConnection(cfg.DatabaseURL)
	if err != nil {
		log.Fatalf("Failed to connect to database: %v", err)
	}
	defer db.Close()

	// Set Gin mode
	if cfg.Environment == "production" {
		gin.SetMode(gin.ReleaseMode)
	}

	// Initialize router
	r := router.SetupRouter(db, cfg)

	// Start server
	port := os.Getenv("PORT")
	if port == "" {
		port = "8080"
	}

	log.Printf("Server starting on port %s", port)
	if err := r.Run(":" + port); err != nil {
		log.Fatalf("Failed to start server: %v", err)
	}
}
```

backend/internal/config/config.go

```
package config

import (
	"os"
)

type Config struct {
	DatabaseURL  string
	JWTSecret    string
	Environment  string
	FrontendURL  string
	Port         string
}

func Load() *Config {
	return &Config{
		DatabaseURL: getEnv("DATABASE_URL", "postgresql://localhost:5432/seminar_hall_db?sslmode=disable"),
		JWTSecret:    getEnv("JWT_SECRET", "your-secret-key-change-in-production"),
		Environment:  getEnv("ENVIRONMENT", "development"),
		FrontendURL:  getEnv("FRONTEND_URL", "http://localhost:3000"),
		Port:         getEnv("PORT", "8080"),
	}
}

func getEnv(key, defaultValue string) string {
	if value := os.Getenv(key); value != "" {
		return value
	}
```

```
        return defaultValue
}
```

backend/internal/database/database.go

```
package database

import (
        "context"
        "fmt"

        "github.com/jackc/pgx/v5"
        "github.com/jackc/pgx/v5/pgxpool"
)

// NewConnection creates a new PostgreSQL connection pool
func NewConnection(databaseURL string) (*pgxpool.Pool, error) {
        config, err := pgxpool.ParseConfig(databaseURL)
        if err != nil {
                return nil, fmt.Errorf("failed to parse database URL: %w", err)
        }

        // Explicitly set search_path to shbs on every new connection.
        // pgx does NOT honour the search_path URL parameter, so we must do it here.
        config.AfterConnect = func(ctx context.Context, conn *pgx.Conn) error {
                _, err := conn.Exec(ctx, "SET search_path TO shbs")
                return err
        }

        pool, err := pgxpool.NewWithConfig(context.Background(), config)
        if err != nil {
                return nil, fmt.Errorf("failed to create connection pool: %w", err)
        }

        // Test connection
        if err := pool.Ping(context.Background()); err != nil {
                return nil, fmt.Errorf("failed to ping database: %w", err)
        }

        return pool, nil
}
```

backend/internal/router/router.go

```
package router

import (
        "seminar-hall-booking-system/internal/config"
        "seminar-hall-booking-system/internal/handlers"
        "seminar-hall-booking-system/internal/middleware"
        "seminar-hall-booking-system/internal/repositories"
        "seminar-hall-booking-system/internal/services"

        "github.com/gin-gonic/gin"
        "github.com/jackc/pgx/v5/pgxpool"
)

func SetupRouter(db *pgxpool.Pool, cfg *config.Config) *gin.Engine {
        r := gin.Default()

        // CORS middleware
        r.Use(middleware.CORS(cfg.FrontendURL))

        // Health check
        r.GET("/health", func(c *gin.Context) {
                c.JSON(200, gin.H{"status": "ok"})
        })

        // Initialize Repositories
        userRepo := repositories.NewUserRepository(db)
        hallRepo := repositories.NewHallRepository(db)
        requestRepo := repositories.NewRequestRepository(db)
        bookingRepo := repositories.NewBookingRepository(db)
        mgmtRepo := repositories.NewManagementRepository(db)

        // Initialize Services
        authService := services.NewAuthService(userRepo, cfg.JWTSecret)
        hallService := services.NewHallService(hallRepo)
        requestService := services.NewRequestService(requestRepo, hallRepo, userRepo, bookingRepo)
        coordinatorService := services.NewCoordinatorService(requestRepo, userRepo, bookingRepo)
        adminService := services.NewAdminService(requestRepo, bookingRepo)

        // Initialize Handlers
        authHandler := handlers.NewAuthHandler(authService, userRepo)
        hallHandler := handlers.NewHallHandler(hallService)
        requestHandler := handlers.NewRequestHandler(requestService)
        coordinatorHandler := handlers.NewCoordinatorHandler(coordinatorService)
        adminHandler := handlers.NewAdminHandler(adminService)
        mgmtHandler := handlers.NewManagementHandler(mgmtRepo)

        // API routes
        api := r.Group("/api")
```

```
	{
		// Public routes
		public := api.Group("")
		{
			public.GET("/halls", hallHandler.GetAllHalls)
			public.GET("/halls/:id", hallHandler.GetHallByID)
			public.GET("/availability", hallHandler.GetAvailability)
			public.GET("/departments", mgmtHandler.GetDepartments)
			public.GET("/clubs", mgmtHandler.GetClubs)
		}

		// Auth routes
		auth := api.Group("/auth")
		{
			auth.POST("/login", authHandler.Login)
			auth.POST("/logout", authHandler.Logout)

			protectedAuth := auth.Group("")
			protectedAuth.Use(middleware.AuthMiddleware(cfg.JWTSecret))
			protectedAuth.GET("/me", authHandler.GetMe)
			protectedAuth.PATCH("/password", authHandler.ChangePassword)
		}

		// Protected routes
		protected := api.Group("")
		protected.Use(middleware.AuthMiddleware(cfg.JWTSecret))
		{
			// Requester routes
			requester := protected.Group("/requests")
			requester.Use(middleware.RequireRole("requester"))
			{
				requester.POST("", requestHandler.CreateRequest)
				requester.GET("/my", requestHandler.GetMyRequests)
			}

			// Dept Coordinator routes
			coordinator := protected.Group("")
			coordinator.Use(middleware.RequireRole("dept_coordinator"))
			{
				coordinator.GET("/requests/dept-pending", coordinatorHandler.GetPendingRequests)
				coordinator.PATCH("/requests/:id/dept-review", coordinatorHandler.ReviewRequest)
				coordinator.GET("/users/faculties", mgmtHandler.GetFaculties)
			}

			// Class management — accessible by both admin and dept_coordinator
			classGroup := protected.Group("")
			classGroup.Use(middleware.RequireRole("admin", "dept_coordinator"))
			{
				classGroup.GET("/classes", mgmtHandler.GetClasses)
				classGroup.POST("/classes", mgmtHandler.CreateClass)
				classGroup.DELETE("/classes/:id", mgmtHandler.DeleteClass)
			}

			// Admin routes
			admin := protected.Group("")
			admin.Use(middleware.RequireRole("admin"))
			{
				// Request & Booking management
				admin.GET("/requests/admin-pending", adminHandler.GetPendingRequests)
				admin.PATCH("/requests/:id/admin-review", adminHandler.ReviewRequest)
				admin.GET("/bookings", adminHandler.GetAllBookings)
				admin.PATCH("/bookings/:id/cancel", mgmtHandler.CancelBooking)
				// Department management
				admin.POST("/departments", mgmtHandler.CreateDepartment)
				admin.PUT("/departments/:id", mgmtHandler.UpdateDepartment)
				admin.DELETE("/departments/:id", mgmtHandler.DeleteDepartment)
				// Hall management
				admin.POST("/halls", mgmtHandler.CreateHall)
				admin.PUT("/halls/:id", mgmtHandler.UpdateHall)
				admin.DELETE("/halls/:id", mgmtHandler.DeleteHall)
				// User management
				admin.GET("/users", mgmtHandler.GetAllUsers)
				admin.POST("/users", mgmtHandler.CreateUser)
				admin.DELETE("/users/:id", mgmtHandler.DeactivateUser)
				admin.PATCH("/users/:id/reactivate", mgmtHandler.ReactivateUser)
				admin.PATCH("/users/:id/password", mgmtHandler.UpdateUserPassword)
				// Club management (write-only; GET /clubs is public)
				admin.POST("/clubs", mgmtHandler.CreateClub)
				admin.DELETE("/clubs/:id", mgmtHandler.DeleteClub)
				// Reports
				admin.GET("/reports/hall-usage", mgmtHandler.GetHallUsageReport)
				admin.GET("/reports/department-usage", mgmtHandler.GetDeptUsageReport)
			}
		}
	}

	return r
}
```

backend/internal/middleware/auth.go

```
package middleware

import (
	"net/http"
	"strings"

	"github.com/gin-gonic/gin"
	"github.com/golang-jwt/jwt/v5"
)

// AuthMiddleware validates JWT token from cookie or Authorization header
func AuthMiddleware(jwtSecret string) gin.HandlerFunc {
	return func(c *gin.Context) {
		// Try to get token from cookie first
		tokenString, err := c.Cookie("auth_token")
		if err != nil {
			// Try Authorization header
			authHeader := c.GetHeader("Authorization")
			if authHeader == "" {
				c.JSON(http.StatusUnauthorized, gin.H{"error": "Unauthorized"})
				c.Abort()
				return
			}

			// Extract token from "Bearer <token>"
			parts := strings.Split(authHeader, " ")
			if len(parts) != 2 || parts[0] != "Bearer" {
				c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid authorization header"})
				c.Abort()
				return
			}
			tokenString = parts[1]
		}

		// Parse and validate token
		token, err := jwt.Parse(tokenString, func(token *jwt.Token) (interface{}, error) {
			if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
				return nil, jwt.ErrSignatureInvalid
			}
			return []byte(jwtSecret), nil
		})

		if err != nil || !token.Valid {
			c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid token"})
			c.Abort()
			return
		}

		// Extract claims
		if claims, ok := token.Claims.(jwt.MapClaims); ok {
			c.Set("user_id", int(claims["user_id"].(float64)))
			c.Set("username", claims["username"].(string))
			c.Set("role", claims["role"].(string))
		}

		c.Next()
	}
}

// RequireRole checks if user has one of the required roles
func RequireRole(allowedRoles ...string) gin.HandlerFunc {
	return func(c *gin.Context) {
		userRole, exists := c.Get("role")
		if !exists {
			c.JSON(http.StatusForbidden, gin.H{"error": "Insufficient permissions"})
			c.Abort()
			return
		}
		for _, r := range allowedRoles {
			if userRole.(string) == r {
				c.Next()
				return
			}
		}
		c.JSON(http.StatusForbidden, gin.H{"error": "Insufficient permissions"})
		c.Abort()
	}
}
```

backend/internal/models/user.go

```
package models

import (
	"time"
)

type UserRole string

const (
```

```
        RoleAdmin           UserRole = "admin"
        RoleDeptCoordinator UserRole = "dept_coordinator"
        RoleRequester       UserRole = "requester"
        RoleFaculty         UserRole = "faculty"
)

type User struct {
        UserID      int       `json:"user_id"`
        Username    string    `json:"username"`
        PasswordHash string   `json:"-"` // Never return in JSON
        FullName    string    `json:"full_name"`
        Email       *string   `json:"email"`
        Phone       *string   `json:"phone"`
        Role        UserRole  `json:"role"`
        DeptID      *int      `json:"dept_id"`
        IsActive    bool      `json:"is_active"`
        CreatedAt   time.Time `json:"created_at"`
        UpdatedAt   time.Time `json:"updated_at"`
}

type RequesterType string

const (
        RequesterTypeClass RequesterType = "class"
        RequesterTypeClub  RequesterType = "club"
)

type Requester struct {
        RequesterID   int           `json:"requester_id"`
        UserID        int           `json:"user_id"`
        RequesterType RequesterType `json:"requester_type"`
        ClassID       *int          `json:"class_id"`
        ClubID        *int          `json:"club_id"`
        CreatedAt     time.Time     `json:"created_at"`
}
```

backend/internal/models/booking.go (excerpt: L1–73)

```
package models

import (
        "time"
)

type ApprovalStatus string

const (
        StatusPending   ApprovalStatus = "pending"
        StatusForwarded ApprovalStatus = "forwarded"
        StatusApproved  ApprovalStatus = "approved"
        StatusRejected  ApprovalStatus = "rejected"
        StatusAmended   ApprovalStatus = "amended"
)

type BookingStatus string

const (
        BookingStatusConfirmed BookingStatus = "confirmed"
        BookingStatusCancelled BookingStatus = "cancelled"
        BookingStatusCompleted BookingStatus = "completed"
        BookingStatusNoShow    BookingStatus = "no_show"
)

type Hall struct {
        HallID     int                    `json:"hall_id"`
        HallName   string                 `json:"hall_name"`
        Capacity   int                    `json:"capacity"`
        Location   *string                `json:"location"`
        Facilities map[string]interface{} `json:"facilities"`
        IsActive   bool                   `json:"is_active"`
}

type Request struct {
        RequestID         int            `json:"request_id"`
        RequesterID       int            `json:"requester_id"`
        HallID            int            `json:"hall_id"`
        HallName          *string        `json:"hall_name,omitempty"`
        HallLocation      *string        `json:"hall_location,omitempty"`
        HallCapacity      *int           `json:"hall_capacity,omitempty"`
        EventTitle        string         `json:"event_title"`
        EventDescription  *string        `json:"event_description"`
        EventDate         time.Time      `json:"event_date"`
        StartTime         string         `json:"start_time"` // TIME format
        EndTime           string         `json:"end_time"`   // TIME format
        ExpectedAttendees *int           `json:"expected_attendees"`
        Purpose           *string        `json:"purpose"`
        DeptStatus        ApprovalStatus `json:"dept_status"`
        DeptApprovedBy    *int           `json:"dept_approved_by"`
        DeptApprovedAt    *time.Time     `json:"dept_approved_at"`
        DeptRemarks       *string        `json:"dept_remarks"`
        AdminStatus       ApprovalStatus `json:"admin_status"`
```

```
        AdminApprovedBy   *int           `json:"admin_approved_by"`
        AdminApprovedAt   *time.Time     `json:"admin_approved_at"`
        AdminRemarks      *string        `json:"admin_remarks"`
        RequestedAt       time.Time      `json:"requested_at"`
        IsCancelled       bool           `json:"is_cancelled"`
}

type Booking struct {
        BookingID   int           `json:"booking_id"`
        RequestID   int           `json:"request_id"`
        HallID      int           `json:"hall_id"`
        RequesterID int           `json:"requester_id"`
        EventTitle  string        `json:"event_title"`
        EventDate   time.Time     `json:"event_date"`
        StartTime   string        `json:"start_time"`
        EndTime     string        `json:"end_time"`
        Status      BookingStatus `json:"status"`
        CreatedAt   time.Time     `json:"created_at"`
        CompletedAt *time.Time    `json:"completed_at"`
}
// ...
```

backend/internal/handlers/auth_handler.go (excerpt: L1–87)

```
package handlers

import (
        "errors"
        "net/http"
        "seminar-hall-booking-system/internal/repositories"
        "seminar-hall-booking-system/internal/services"
        "strings"

        "github.com/gin-gonic/gin"
        "github.com/jackc/pgx/v5"
)

type AuthHandler struct {
        authService *services.AuthService
        userRepo    *repositories.UserRepository
}

func NewAuthHandler(authService *services.AuthService, userRepo *repositories.UserRepository) *AuthHandler {
        return &AuthHandler{
                authService: authService,
                userRepo:    userRepo,
        }
}

type LoginRequest struct {
        Username string `json:"username" binding:"required"`
        Password string `json:"password" binding:"required"`
}

// Login handles user authentication
func (h *AuthHandler) Login(c *gin.Context) {
        var req LoginRequest
        if err := c.ShouldBindJSON(&req); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        token, user, err := h.authService.Login(c.Request.Context(), req.Username, req.Password)
        if err != nil {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid credentials"})
                return
        }

        // Set HTTP-only cookie for the JWT
        c.SetCookie("auth_token", token, 3600*24*7, "/", "", false, true)

        c.JSON(http.StatusOK, gin.H{
                "message": "success",
                "token":   token,
                "user":    user,
        })
}

// Logout clears the auth cookie
func (h *AuthHandler) Logout(c *gin.Context) {
        c.SetCookie("auth_token", "", -1, "/", "", false, true)
        c.JSON(http.StatusOK, gin.H{"message": "logged out"})
}

// GetMe returns the authenticated user's details
func (h *AuthHandler) GetMe(c *gin.Context) {
        userID, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }
```

```
        // Fetch fresh user details from DB
        user, err := h.userRepo.GetByID(c.Request.Context(), userID.(int))
        if err != nil || user == nil {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "user not found"})
                return
        }

        // Check if user is a requester and fetch requester info
        // if user.Role == models.RoleRequester
        if user.Role == "requester" {
                reqInfo, err := h.userRepo.GetRequesterByUserID(c.Request.Context(), user.UserID)
                if err == nil && reqInfo != nil {
                        c.JSON(http.StatusOK, gin.H{"user": user, "requester": reqInfo})
                        return
                }
        }

        c.JSON(http.StatusOK, gin.H{"user": user})
}
// ...
```

backend/internal/services/auth_service.go (excerpt: L1–63)

```
package services

import (
        "context"
        "errors"
        "fmt"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/repositories"
        "time"

        "github.com/golang-jwt/jwt/v5"
        "golang.org/x/crypto/bcrypt"
)

// ErrInvalidCurrentPassword is returned when ChangeOwnPassword verification fails.
var ErrInvalidCurrentPassword = errors.New("invalid current password")

type AuthService struct {
        userRepo  *repositories.UserRepository
        jwtSecret string
}

func NewAuthService(userRepo *repositories.UserRepository, jwtSecret string) *AuthService {
        return &AuthService{
                userRepo:  userRepo,
                jwtSecret: jwtSecret,
        }
}

func (s *AuthService) Login(ctx context.Context, username, password string) (string, *models.User, error) {
        user, err := s.userRepo.GetByUsername(ctx, username)
        if err != nil {
                return "", nil, err
        }
        if user == nil {
                return "", nil, errors.New("invalid credentials")
        }

        // Attempt bcrypt comparison
        err = bcrypt.CompareHashAndPassword([]byte(user.PasswordHash), []byte(password))
        if err != nil {
                // Fallback for sample DB data which might be plain text or pseudo-hashed strings like
"hashedpass_admin"
                if user.PasswordHash != password {
                        return "", nil, errors.New("invalid credentials")
                }
        }

        // Generate JWT claims
        token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
                "user_id":  user.UserID,
                "username": user.Username,
                "role":     user.Role,
                "exp":      time.Now().Add(time.Hour * 24 * 7).Unix(), // 7 days expiration
        })

        // Sign token
        tokenString, err := token.SignedString([]byte(s.jwtSecret))
        if err != nil {
                return "", nil, err
        }

        return tokenString, user, nil
}
// ...
```

backend/internal/handlers/request_handler.go (excerpt: L1–85)

```
package handlers

import (
```

```
        "fmt"
        "net/http"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/services"
        "time"

        "github.com/gin-gonic/gin"
)

type RequestHandler struct {
        requestService *services.RequestService
}

func NewRequestHandler(requestService *services.RequestService) *RequestHandler {
        return &RequestHandler{requestService: requestService}
}

type CreateRequestPayload struct {
        HallID            int     `json:"hall_id" binding:"required"`
        EventTitle        string  `json:"event_title" binding:"required"`
        EventDescription  *string `json:"event_description"`
        EventDate         string  `json:"event_date" binding:"required"` // YYYY-MM-DD
        StartTime         string  `json:"start_time" binding:"required"` // HH:MM
        EndTime           string  `json:"end_time" binding:"required"`   // HH:MM
        ExpectedAttendees *int    `json:"expected_attendees"`
        Purpose           *string `json:"purpose"`
}

// CreateRequest creates a new booking request for a student/club
func (h *RequestHandler) CreateRequest(c *gin.Context) {
        userIDStr, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }
        userID := userIDStr.(int)

        var payload CreateRequestPayload
        if err := c.ShouldBindJSON(&payload); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        // Parse date
        eventDate, err := time.Parse("2006-01-02", payload.EventDate)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": "invalid event_date format (use YYYY-MM-DD)"})
                return
        }

        // Format times string slightly (adding seconds if omitted by client)
        startTime := payload.StartTime
        if len(startTime) == 5 {
                startTime += ":00"
        }
        endTime := payload.EndTime
        if len(endTime) == 5 {
                endTime += ":00"
        }

        req := models.Request{
                HallID:            payload.HallID,
                EventTitle:        payload.EventTitle,
                EventDescription:  payload.EventDescription,
                EventDate:         eventDate,
                StartTime:         startTime,
                EndTime:           endTime,
                ExpectedAttendees: payload.ExpectedAttendees,
                Purpose:           payload.Purpose,
        }

        err = h.requestService.CreateRequest(c.Request.Context(), userID, &req)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        c.JSON(http.StatusCreated, gin.H{
                "message": "request submitted successfully",
                "data":    req,
        })
}
// ...
```

backend/internal/services/request_service.go (excerpt: L1–86)

```
package services

import (
        "context"
        "errors"
        "seminar-hall-booking-system/internal/models"
```

```
        "seminar-hall-booking-system/internal/repositories"
        "time"
)

type RequestService struct {
        requestRepo *repositories.RequestRepository
        hallRepo    *repositories.HallRepository
        userRepo    *repositories.UserRepository
        bookingRepo *repositories.BookingRepository
}

func NewRequestService(
        reqRepo *repositories.RequestRepository,
        hallRepo *repositories.HallRepository,
        userRepo *repositories.UserRepository,
        bookingRepo *repositories.BookingRepository,
) *RequestService {
        return &RequestService{
                requestRepo: reqRepo,
                hallRepo:    hallRepo,
                userRepo:    userRepo,
                bookingRepo: bookingRepo,
        }
}

// CreateRequest handles the business logic for submitting a new seminar hall request
func (s *RequestService) CreateRequest(ctx context.Context, userID int, req *models.Request) error {
        // 1. Verify user is a requester
        requester, err := s.userRepo.GetRequesterByUserID(ctx, userID)
        if err != nil {
                return err
        }
        if requester == nil {
                return errors.New("user is not registered as a requester (class or club)")
        }

        // 2. Validate basic rules
        if req.ExpectedAttendees != nil && *req.ExpectedAttendees <= 0 {
                return errors.New("expected attendees must be greater than zero")
        }

        // 3. Verify Hall exists and capacity is sufficient
        hall, err := s.hallRepo.GetByID(ctx, req.HallID)
        if err != nil {
                return err
        }
        if hall == nil {
                return errors.New("hall not found")
        }
        if req.ExpectedAttendees != nil && *req.ExpectedAttendees > hall.Capacity {
                return errors.New("expected attendees exceeds hall capacity")
        }

        // 4. Validate time (end_time > start_time)
        startTime, err := time.Parse("15:04:05", req.StartTime)
        if err != nil {
                return errors.New("invalid start_time format, expected HH:MM:SS")
        }
        endTime, err := time.Parse("15:04:05", req.EndTime)
        if err != nil {
                return errors.New("invalid end_time format, expected HH:MM:SS")
        }
        if !endTime.After(startTime) {
                return errors.New("end time must be strictly after start time")
        }

        // 5. Check for overlapping bookings before placing the request
        overlap, err := s.bookingRepo.CheckOverlap(ctx, req.HallID, req.EventDate, req.StartTime, req.EndTime)
        if err != nil {
                return err
        }
        if overlap {
                return errors.New("cannot place request: this time slot overlaps with an existing confirmed
booking")
        }

        // 6. Build and save request
        req.RequesterID = requester.RequesterID

        return s.requestRepo.Create(ctx, req)
}
// ...
```

backend/internal/repositories/request_repository.go (excerpt: L1–40, L145–153)

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"

        "github.com/jackc/pgx/v5"
```

```
        "github.com/jackc/pgx/v5/pgxpool"
)

type RequestRepository struct {
        db *pgxpool.Pool
}

func NewRequestRepository(db *pgxpool.Pool) *RequestRepository {
        return &RequestRepository{db: db}
}

// Create new request
func (r *RequestRepository) Create(ctx context.Context, req *models.Request) error {
        query := `
                INSERT INTO requests (requester_id, hall_id, event_title, event_description, event_date,
start_time, end_time, expected_attendees, purpose)
                VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
                RETURNING request_id, dept_status, admin_status, requested_at
        `

        err := r.db.QueryRow(ctx, query,
                req.RequesterID,
                req.HallID,
                req.EventTitle,
                req.EventDescription,
                req.EventDate,
                req.StartTime,
                req.EndTime,
                req.ExpectedAttendees,
                req.Purpose,
        ).Scan(&req.RequestID, &req.DeptStatus, &req.AdminStatus, &req.RequestedAt)

        return err
}
// ...
func (r *RequestRepository) UpdateDeptStatus(ctx context.Context, reqID int, status models.ApprovalStatus, byUserID
int, remarks *string) error {
        query := `
                UPDATE requests
                SET dept_status = $1, dept_approved_by = $2, dept_approved_at = NOW(), dept_remarks = $3
                WHERE request_id = $4
        `
        _, err := r.db.Exec(ctx, query, status, byUserID, remarks, reqID)
        return err
}
// ...
```

backend/internal/handlers/coordinator_handler.go

```
package handlers

import (
        "net/http"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/services"
        "strconv"

        "github.com/gin-gonic/gin"
)

type CoordinatorHandler struct {
        coordinatorService *services.CoordinatorService
}

func NewCoordinatorHandler(coordinatorService *services.CoordinatorService) *CoordinatorHandler {
        return &CoordinatorHandler{coordinatorService: coordinatorService}
}

// GetPendingRequests gets requests waiting for department approval
func (h *CoordinatorHandler) GetPendingRequests(c *gin.Context) {
        userIDStr, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }
        userID := userIDStr.(int)

        requests, err := h.coordinatorService.GetDepartmentPendingRequests(c.Request.Context(), userID)
        if err != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
                return
        }

        if requests == nil {
                requests = []models.Request{}
        }

        c.JSON(http.StatusOK, requests)
}

type ReviewRequestPayload struct {
        Action  string  `json:"action" binding:"required"` // "approve" or "reject"
```

```
        Remarks *string `json:"remarks"`
}

// ReviewRequest approves or rejects a department request
func (h *CoordinatorHandler) ReviewRequest(c *gin.Context) {
        userIDStr, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }
        userID := userIDStr.(int)

        requestIDStr := c.Param("id")
        requestID, err := strconv.Atoi(requestIDStr)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": "invalid request id"})
                return
        }

        var payload ReviewRequestPayload
        if err := c.ShouldBindJSON(&payload); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        err = h.coordinatorService.ReviewRequest(c.Request.Context(), userID, requestID, payload.Action,
payload.Remarks)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        c.JSON(http.StatusOK, gin.H{"message": "request " + payload.Action + " successfully"})
}
```

backend/internal/services/coordinator_service.go (excerpt: L1–22, L40–79)

```
package services

import (
        "context"
        "errors"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/repositories"
)

type CoordinatorService struct {
        requestRepo *repositories.RequestRepository
        userRepo    *repositories.UserRepository
        bookingRepo *repositories.BookingRepository
}

func NewCoordinatorService(reqRepo *repositories.RequestRepository, userRepo *repositories.UserRepository,
bookingRepo *repositories.BookingRepository) *CoordinatorService {
        return &CoordinatorService{
                requestRepo: reqRepo,
                userRepo:    userRepo,
                bookingRepo: bookingRepo,
        }
}
// ...
// ReviewRequest allows the coordinator to approve (forward) or reject a request
func (s *CoordinatorService) ReviewRequest(ctx context.Context, coordinatorID int, requestID int, action string,
remarks *string) error {
        // 1. Verify coordinator
        coordinator, err := s.userRepo.GetByID(ctx, coordinatorID)
        if err != nil || coordinator == nil || coordinator.DeptID == nil {
                return errors.New("invalid coordinator")
        }

        // 2. Verify request exists and is pending
        req, err := s.requestRepo.GetByID(ctx, requestID)
        if err != nil || req == nil {
                return errors.New("request not found")
        }

        if req.DeptStatus != models.StatusPending {
                return errors.New("request is no longer pending department approval")
        }

        // In a full implementation, verify the request belongs to the coordinator's department here by looking up
the requester's department.

        // 3. Update status
        var newStatus models.ApprovalStatus
        if action == "approve" {
                // Before forwarding, check if there's an existing overlapping booking
                overlap, err := s.bookingRepo.CheckOverlap(ctx, req.HallID, req.EventDate, req.StartTime,
req.EndTime)
                if err != nil {
                        return err
                }
```

```
                if overlap {
                        return errors.New("cannot forward: this request overlaps with an existing confirmed
booking")
                }
                newStatus = models.StatusForwarded
        } else if action == "reject" {
                newStatus = models.StatusRejected
        } else {
                return errors.New("invalid action. Must be 'approve' or 'reject'")
        }

        return s.requestRepo.UpdateDeptStatus(ctx, requestID, newStatus, coordinatorID, remarks)
}
```

backend/internal/repositories/booking_repository.go (excerpt: L1–18, L126–138, L140–145)

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"
        "time"

        "github.com/jackc/pgx/v5/pgxpool"
)

type BookingRepository struct {
        db *pgxpool.Pool
}

func NewBookingRepository(db *pgxpool.Pool) *BookingRepository {
        return &BookingRepository{db: db}
}

// ...
// CheckOverlap returns true if there's an existing active booking for the same hall and time
func (r *BookingRepository) CheckOverlap(ctx context.Context, hallID int, date time.Time, startTime string, endTime
string) (bool, error) {
        query := `
                SELECT COUNT(*)
                FROM bookings
                WHERE hall_id = $1
                  AND event_date = $2
                  AND status != 'cancelled'
                  AND (
                          (start_time < $4 AND end_time > $3)
                  )
        `
        var count int
// ...
        err := r.db.QueryRow(ctx, query, hallID, date, startTime, endTime).Scan(&count)
        if err != nil {
                return false, err
        }
        return count > 0, nil
}
```